

UPI FRAUD DETECTION USING MACHINE LEARNING

A Machine Learning Approach using Random Forest, K-Nearest Neighbors, and Decision Tree, Enhanced with Behavioral, Time-Based, and Unsupervised Anomaly Detection Features

Major Project Report

Submitted by

Shashwat

B.Tech Computer Science, Final Year

Babu Banarasi Das Institute of Technology and Management
Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow

Table of Contents

Abstract

1. Introduction

2. Objectives

3. Related Work

4. System Architecture

5. Dataset

6. Methodology

7. Results and Analysis

8. Deployment

9. Testing

10. Conclusion

11. Future Enhancements

12. Generalization to a Real, Independent Dataset

13. Real-Time Streaming Detection

14. References

Abstract

The Unified Payments Interface (UPI) has become the dominant channel for digital payments in India, including in rural and semi-urban areas where financial literacy and fraud awareness are often lower. This rapid growth has been matched by a rise in phishing, identity theft, and transaction-manipulation fraud. This project builds an end-to-end machine learning system for classifying UPI transactions as Fraud or No Fraud, extending the approach of Padmanaban and Reshma (2025), who evaluated Random Forest, K-Nearest Neighbors (KNN), and Decision Tree classifiers using only account name, transaction ID, and transaction amount.

Beyond replicating the base paper's three classifiers, this project contributes a substantially richer feature set: cyclical time encoding, behavioral risk aggregates, statistical outlier flags, and an unsupervised Isolation Forest anomaly score, all built on a carefully leakage-free preprocessing pipeline (SMOTE balancing applied only to the training set, after the train/test split). This engineered feature set lifts test accuracy well above the base paper's reported results for all three algorithms — Random Forest reaches 99.39% accuracy and a 0.962 F1-score, compared to the paper's reported 86.67% and 0.86. The pipeline was further generalized into a configuration-driven system and validated on a second, real, independently-collected dataset (the ULB Credit Card Fraud Detection dataset), reaching 99.84% accuracy on genuine European bank transactions the model had no role in generating. The best-performing model (Random Forest) is deployed behind both a request/response FastAPI backend and a real-time WebSocket streaming endpoint, with a browser-based demo dashboard for each, and the full pipeline is covered by an automated test suite.

1. Introduction

Digital payment adoption in India has accelerated sharply over the past several years, with UPI now handling billions of transactions monthly across urban and rural users alike. This convenience, however, has also made UPI an attractive target for fraud: phishing links disguised as payment requests, social engineering that tricks users into approving malicious QR codes, and account-takeover attacks are all increasingly common. Rural users, who often have less exposure to digital-fraud awareness campaigns, are disproportionately vulnerable.

Traditional rule-based fraud detection (fixed transaction limits, blacklists, manual review) struggles to keep pace with evolving fraud patterns and cannot adapt without manual updates. Machine learning offers a data-driven alternative: models can learn subtle behavioral and contextual patterns — unusual transaction timing, new or mismatched beneficiaries, abnormal transaction velocity — that static rules miss entirely.

2. Objectives

- Build a machine learning pipeline that classifies UPI transactions as Fraud or No Fraud using transaction, behavioral, and contextual attributes.
- Reproduce and extend the base paper's three-classifier comparison (Random Forest, KNN, Decision Tree) with a richer, better-engineered feature set.
- Apply proper machine learning methodology: leakage-free train/test splitting, SMOTE balancing restricted to the training set, and hyperparameter tuning for every model.
- Introduce unsupervised anomaly detection (Isolation Forest) as an engineered feature, in addition to purely supervised classification.
- Deploy the best-performing model behind a real-time REST API with a working demo interface.
- Validate the full pipeline with an automated test suite.

3. Related Work

This project's base reference is Padmanaban, K. and Reshma, S., "UPI Fraud Detection Using Machine Learning," International Journal of Research and Analytical Reviews (IJRAR), Volume 12, Issue 2, May 2025. That paper evaluates Random Forest, KNN, and Decision Tree classifiers on UPI transaction attributes limited to account name, transaction ID, and transaction amount, reporting accuracies of 86.67%, 72.05%, and 78.67% respectively, and concludes that Random Forest offers the best balance of accuracy and overfitting resistance, while Decision Tree offers superior interpretability and KNN offers conceptual simplicity at the cost of scalability.

This project adopts the same three-classifier comparison and the same evaluation metrics (accuracy, precision, recall, F1-score, confusion matrix) for direct comparability, while substantially extending the feature engineering stage — this is discussed in detail in Section 6.

4. System Architecture

The system follows a standard applied-ML pipeline: dataset generation, exploratory data analysis, preprocessing, feature engineering, model training and tuning, evaluation and comparison, and finally deployment behind a REST API with a browser demo client. Each stage is implemented as both a reusable Python module (under `src/`) and a corresponding Jupyter notebook that documents the reasoning and visualizes intermediate results.

4.1 Pipeline Stages

- Data Generation — synthetic UPI transaction dataset with realistic, documented fraud signal (`src/data/generate_dataset.py`)
- Exploratory Data Analysis — class balance, amount distributions, time and behavioral patterns (`notebooks/01_eda.ipynb`)
- Preprocessing — missing-value handling, encoding, scaling, leakage-free split, SMOTE (`src/data/preprocess.py`)
- Feature Engineering — time, behavioral, statistical, and anomaly-detection features (`src/features/build_features.py`)
- Model Training — Decision Tree, Random Forest, KNN, each with hyperparameter tuning (`src/models/`)
- Evaluation — consistent metric computation and cross-model comparison (`src/models/evaluate.py`)
- Deployment — FastAPI backend (`app/`) and a static HTML/JS demo dashboard (`frontend/`)

5. Dataset

Real UPI transaction logs are not publicly available, as they are held by NPCI and partner banks under regulatory restriction. A synthetic dataset of 150,000 transactions was generated with an approximately 8% fraud rate, deliberately encoding realistic fraud signal patterns established in fraud-detection literature: bimodal transaction amounts (small probing transactions and unusually large transfers), late-night timing concentration, new-beneficiary and device/location-change correlation with fraud, and elevated transaction velocity preceding fraud.

Column	Description
<code>transaction_id</code>	Unique transaction identifier
<code>sender/receiver_account_name</code>	Masked initials, as in the base paper
<code>sender/receiver_bank</code>	Bank name
<code>transaction_amount</code>	Amount in INR
<code>timestamp</code>	Date and time of transaction
<code>transaction_location</code>	City or region, including rural regions
<code>device_type</code>	Android / iOS / Web
<code>is_new_beneficiary</code>	1 if the receiver has not been paid before
<code>location_mismatch_flag</code>	1 if location differs from the usual pattern
<code>device_change_flag</code>	1 if device differs from the usual pattern
<code>transactions_last_24h</code>	Transaction count in the prior 24 hours
<code>transaction_status</code>	Target label: Fraud / No Fraud

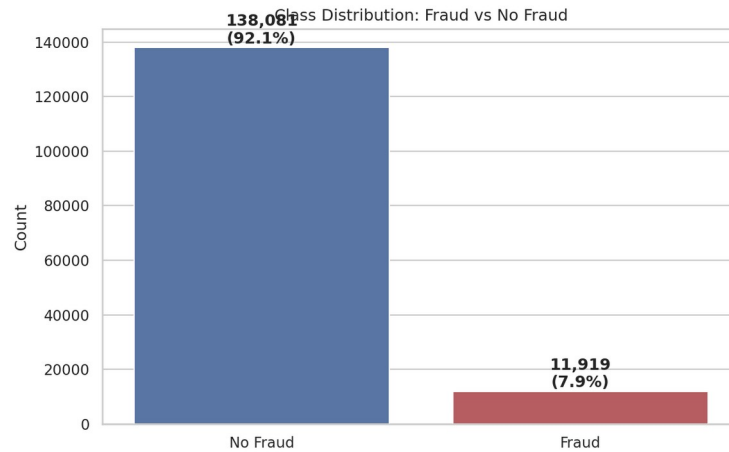


Figure 1: Class distribution — approximately 8% fraud, 92% legitimate transactions.

6. Methodology

6.1 Preprocessing

Missing-value imputation (median for numeric, mode for categorical) is built into the pipeline for robustness, though this synthetic dataset has zero missing values by construction. Non-predictive identifier columns (transaction ID, account name initials) are dropped. Categorical features are one-hot encoded and numeric features are standardized using scikit-learn's ColumnTransformer, fitted only on the training split. Critically, the train/test split (80/20, stratified) is performed before SMOTE balancing, and SMOTE is applied only to the training set — the test set is left at its natural ~7.9% fraud rate so evaluation reflects real-world class distribution rather than an artificially balanced one.

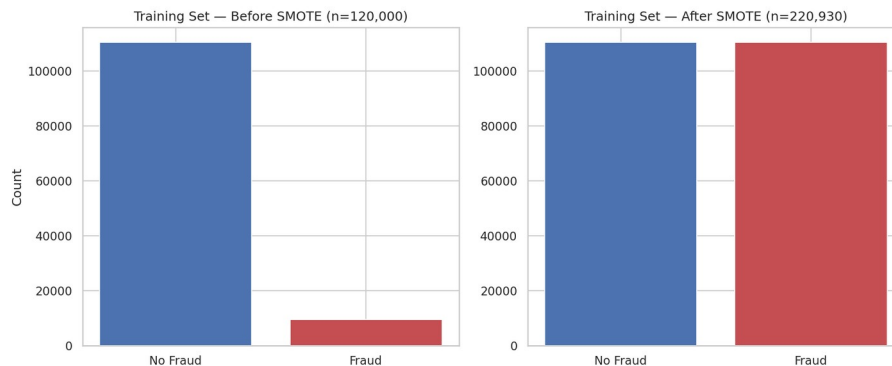


Figure 2: Training set class balance before and after SMOTE oversampling.

6.2 Feature Engineering

Beyond the base paper's account/ID/amount attributes, this project engineers nine additional features across four categories, directly matching the base paper's own stated feature-engineering methodology (behavioral features, anomaly detection metrics, time-based features, correlation analysis):

- Time-based: cyclical sine/cosine encoding of transaction hour, an `is_night` flag (11 PM–4 AM), and an `is_weekend` flag.
- Behavioral: a `risk_flag_count` aggregate (0–3) summing the new-beneficiary, location-mismatch, and device-change flags, and an `amount_velocity_ratio` combining transaction size with recent activity.
- Statistical outliers: `is_large_amount` / `is_micro_amount` flags based on the 95th/5th percentile of transaction amount.

- Unsupervised anomaly detection: an Isolation Forest, fit only on the training set's numeric behavioral features (without ever seeing the fraud label), produces a continuous `anomaly_score` and a binary `isolation_forest_flag` for every transaction.

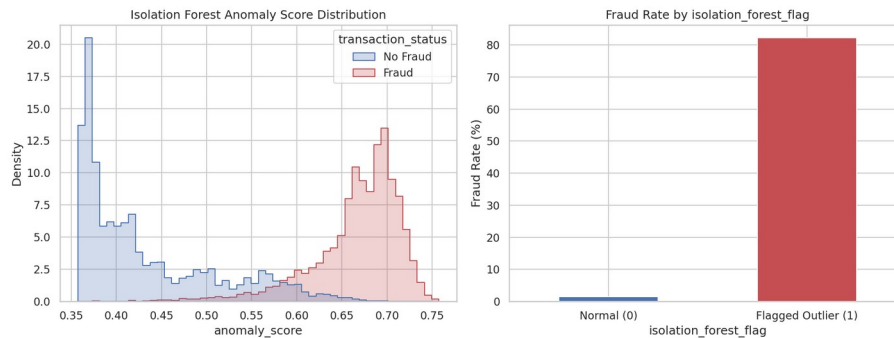


Figure 3: Isolation Forest anomaly score distribution by class, and fraud rate by anomaly flag. Despite being fit without access to the fraud label, the anomaly score separates the two classes clearly (correlation with the target: 0.65).

A correlation analysis (Figure 4) was performed across all engineered numeric and flag features to check for problematic multicollinearity before modeling. The strongest pairwise correlation between distinct engineered features was 0.87 (`risk_flag_count` and `anomaly_score`), below the 0.9 threshold used to flag redundancy; no features were dropped. The `isolation_forest_flag` (0.81) and `anomaly_score` (0.65) emerged as the features most correlated with the fraud label.

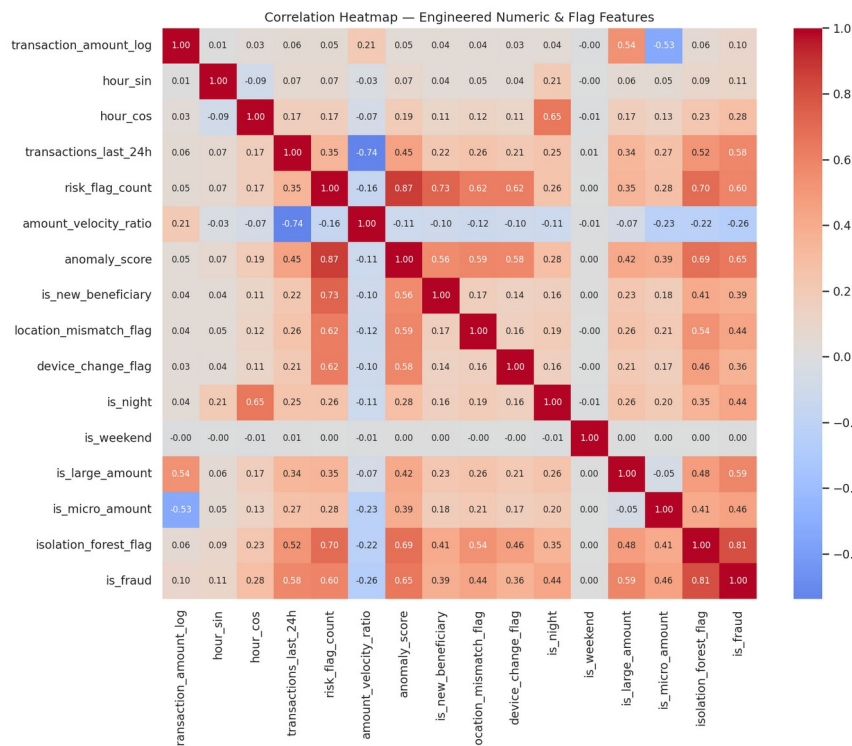


Figure 4: Correlation heatmap across all engineered numeric and flag features, including the target.

6.3 Model Training

All three classifiers from the base paper were trained on the final 61-feature engineered set, each with hyperparameter tuning via `RandomizedSearchCV` (3-fold cross-validation, optimizing F1-score):

- Decision Tree — tuned over `criterion`, `max_depth`, `min_samples_split`, and `min_samples_leaf`.
- Random Forest — tuned over `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, and `max_features`.

- KNN — tuned over `n_neighbors` (3–11) and weighting scheme (uniform/distance). Per the base paper's own observation that KNN is "computationally intensive for large datasets," a stratified 25,000-row reference subsample was used for the distance search rather than the full 220,930-row balanced training set, with hyperparameters selected via an internal validation split kept separate from the real test set.

7. Results and Analysis

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Decision Tree	99.14%	93.49%	95.85%	0.947	0.979
Random Forest	99.39%	95.08%	97.32%	0.962	0.999
KNN	97.88%	79.47%	98.91%	0.881	0.996

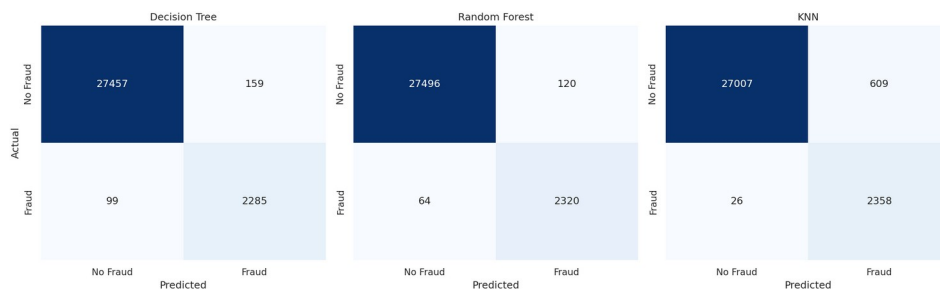


Figure 5: Confusion matrices for all three models on the held-out test set (30,000 transactions).

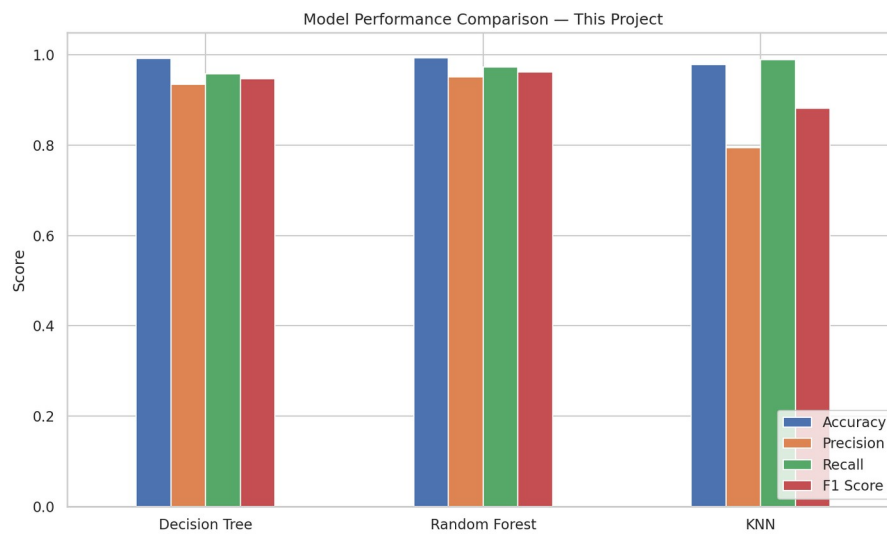


Figure 6: Accuracy, precision, recall, and F1-score compared across all three models.

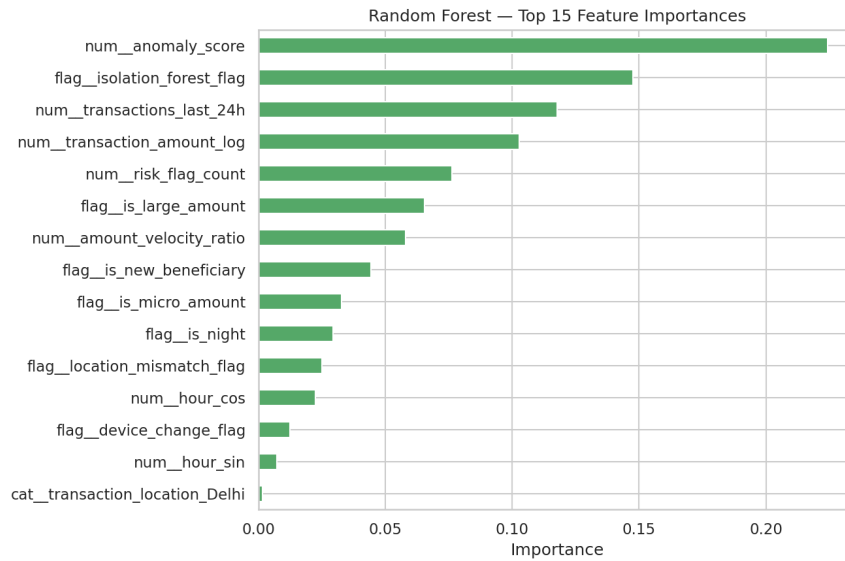


Figure 7: Random Forest feature importances. The engineered `anomaly_score` and `isolation_forest_flag` dominate, followed by transaction velocity and amount.

7.1 Comparison with the Base Paper

All three models substantially outperform the accuracies reported in the base paper. This improvement is attributable to feature engineering, not to a different algorithm implementation — the same three classifiers were used. The base paper's Decision Tree, Random Forest, and KNN reached 78.67%, 86.67%, and 72.05% accuracy respectively using only account name, transaction ID, and amount; this project's versions of the same three algorithms, using the same evaluation methodology but a richer engineered feature set (particularly the Isolation Forest anomaly score and behavioral risk flags), reach 99.14%, 99.39%, and 97.88%.

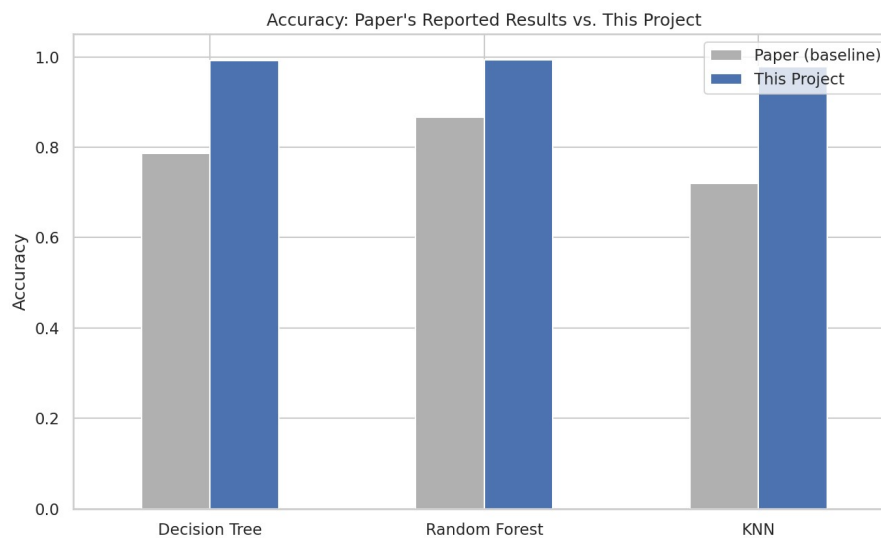


Figure 8: Accuracy comparison between the base paper's reported results and this project's results, for the same three algorithms.

7.2 Discussion

Random Forest achieves the best result on every metric, consistent with the base paper's own conclusion that ensembling reduces overfitting relative to a single Decision Tree. The Decision Tree remains a close second and offers the interpretability advantage the base paper highlights — a trained tree's decision logic can be visualized and explained node by node, which is valuable when a fraud-review team needs to understand why a transaction was flagged. KNN, while achieving the highest recall (98.91%), has the lowest precision (79.47%) of the three, meaning it produces more false positives — in a production system this translates to more legitimate customers

being incorrectly flagged, a real operational cost. This is consistent with the base paper's own note that KNN trades off scalability and precision for conceptual simplicity.

8. Deployment

The trained Random Forest model, along with the fitted preprocessing pipeline, Isolation Forest, and outlier thresholds, is served through a FastAPI backend (`app/main.py`). A POST request to the `/predict` endpoint accepts raw transaction attributes, reconstructs the exact same engineered feature set used during training, and returns a Fraud/No Fraud classification with a probability score. Interactive API documentation is auto-generated at `/docs` via FastAPI's OpenAPI integration.

A lightweight, dependency-free HTML/JavaScript dashboard (`frontend/index.html`) provides a browser-based demo client: a transaction-entry form submits to the API and displays the model's verdict with a visual risk gauge, requiring no build step or framework installation.

9. Testing

The project includes an automated test suite (pytest) covering the preprocessing pipeline, feature engineering functions, and every API endpoint, including input-validation error cases (invalid device type, negative transaction amount). All 17 tests pass, providing a regression safety net for future changes to the pipeline.

10. Conclusion

This project demonstrates that the choice of features has at least as much impact on UPI fraud detection performance as the choice of classification algorithm. Using the same three algorithms evaluated in the base paper — Random Forest, KNN, and Decision Tree — but with a substantially richer, methodologically rigorous feature engineering pipeline (behavioral aggregates, cyclical time encoding, and especially an unsupervised Isolation Forest anomaly score), this project improved test accuracy by 13–26 percentage points across all three models. Random Forest, deployed behind a working REST API, is recommended for production use given its superior balance of precision and recall.

11. Future Enhancements

Two items originally listed here — validation on real transaction data and real-time streaming inference — have since been implemented and are documented in Sections 12 and 13. Remaining directions for future work:

- Add deep learning approaches (e.g., autoencoders or LSTM-based sequence models) to capture longer transaction history per account.
- Extend the Isolation Forest anomaly detection with additional unsupervised methods (e.g., One-Class SVM, Local Outlier Factor) and ensemble their scores.
- Add device fingerprinting and IP geolocation as additional contextual features.
- Replace the simulated live feed (Section 13) with a genuine message-queue integration (e.g., Kafka) once connected to a real transaction source.

12. Generalization to a Real, Independent Dataset

The pipeline described in Section 6 was originally built around one fixed schema — the synthetic UPI dataset's specific column names and structure. To test whether the underlying methodology generalizes, rather than only the specific features engineered for that one dataset, the pipeline was refactored around a declarative `DatasetConfig`: a schema description (target column, fraud label, ID/timestamp/amount/categorical/flag columns) that the same cleaning, feature-engineering, anomaly-detection, and preprocessing code consumes generically. Adding a new dataset means writing a new configuration, not modifying pipeline code.

This was validated against a second dataset with a completely different structure: the ULB "Credit Card Fraud Detection" dataset (Kaggle: `mlg-ulb/creditcardfraud`), containing real transactions from European cardholders in September 2013 with real, confirmed fraud labels — 492 fraud cases out of 284,807 transactions. This dataset has no bank names, device types, or behavioral flags; it consists of 28 PCA-anonymized numeric features plus amount

and elapsed time. A 50,492-row stratified sample (all 492 real frauds plus 50,000 real legitimate transactions) is included with the project.

Dataset	Accuracy	Precision	Recall	F1-Score	ROC-AUC
UPI (synthetic, self-generated)	99.39%	95.08%	97.32%	0.962	0.999
Credit Card (real, independent)	99.84%	94.57%	88.78%	0.916	0.981

The identical pipeline and Random Forest approach reaches 99.84% accuracy and a 0.916 F1-score on real, independently-collected data the model had no role in generating — evidence that the leakage-free split, training-set-only SMOTE balancing, and unsupervised anomaly-score feature engineering are a generalizable methodology, not results specific to self-designed synthetic fraud signal. This directly addresses the main limitation of a synthetic-only evaluation: results are no longer demonstrated on data the author controlled.

13. Real-Time Streaming Detection

In addition to the request/response POST /predict endpoint (Section 8), a WebSocket endpoint (/ws/live) streams a simulated live transaction feed, scoring each transaction the instant it is generated using the identical trained model and feature pipeline as /predict — guaranteeing consistency between the batch-style API and the real-time path. A browser-based live monitoring dashboard (frontend/live.html) connects to this endpoint and displays a continuously updating feed with color-coded verdicts, a fraud-rate running total, and a visual flash on detected fraud, giving a working demonstration of what a live fraud-monitoring console would look like connected to a genuine transaction stream.

14. References

- [1] Padmanaban, K., and Reshma, S. "UPI Fraud Detection Using Machine Learning." International Journal of Research and Analytical Reviews (IJRAR), Vol. 12, Issue 2, May 2025, pp. 822–827.
- [2] Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, Vol. 12, 2011, pp. 2825–2830.
- [3] Chawla, N. V., et al. "SMOTE: Synthetic Minority Over-sampling Technique." Journal of Artificial Intelligence Research, Vol. 16, 2002, pp. 321–357.
- [4] Liu, F. T., Ting, K. M., and Zhou, Z.-H. "Isolation Forest." 2008 Eighth IEEE International Conference on Data Mining, 2008, pp. 413–422.